

BEGUM ROKEYA UNIVERSITY, RANGPUR

DEPARTMENT OF STATISTICS

STATISTICAL DATA ANALYSIS

A Comprehensive Laboratory Report



COURSE DETAILS

STAT-4104

CATEGORICAL DATA ANALYSIS

SUBMITTED BY

Name: Promie Saha
ID: 12110042
Reg No: 000015905
Session: 2021-2022
Year/Sem: 4th Year, 1st Sem

SUBMITTED TO

Name: Siddikur Rahman
Title: Associate Professor
Dept: Statistics
Inst: Begum Rokeya University



Submission: May 17, 2026

☰ CONTENTS

01	Lung Disease Analysis	03
02	One-Way ANOVA - Exercise	08
03	Chi-Square - Child Anemia	10
04	Fisher's Exact - Drug Test	12
05	Logistic - Grad Admission	14
06	Poisson - Awards	16
07	Negative Binomial - Absences	18
08	Zero-Inflated Poisson - Fish	20
09	ZIP - Detailed Analysis	22
10	Zero-Truncated Poisson	24
11	Zero-Truncated NB	26



Comprehensive statistical modeling using GLM frameworks

PROBLEM 1

Lung Disease Dataset Analysis using Python

Problem Overview

Using the lung disease dataset, perform exploratory data analysis and fit a logistic regression model to identify key determinants of lung disease. Interpret your findings and discuss public health implications.

A. Data Import and Initial Exploration

```
1 import pandas as pd
2 import numpy as np
3 from scipy.stats import chi2_contingency, fisher_exact
4 import statsmodels.formula.api as smf
5 from sklearn.metrics import roc_auc_score, confusion_matrix
6 from sklearn.metrics import accuracy_score, recall_score
7
8 df = pd.read_csv('lung_disease.csv')
9 print(df.head())
```

>_ RESULT:

	Age	Smoking	Pollution	Income	LungDisease
0	45	Yes	High	Low	Yes
1	32	No	Low	High	No
2	58	Yes	High	Low	Yes
3	27	No	Low	High	No
4	61	Yes	High	Low	Yes

B. Exploratory Analysis

Age Distribution

```
1 print(df['Age'].describe())
2 print('Skewness =', df['Age'].skew())
```

>_ RESULT:

```
count    500.000000
mean     43.904000
std      14.604841
min      20.000000
25%      30.750000
50%      45.000000
75%      56.000000
max      69.000000
```

Skewness = 0.012

Key Insight

The age distribution is nearly perfectly normal with skewness of 0.012, indicating a representative sample across all adult age groups.

Smoking Prevalence

```
1 smoking_prop = df['Smoking'].value_counts(normalize=True) *
  100
2 print(smoking_prop)
```

>_ RESULT:

```
No      58.6%
Yes     41.4%
```

Income Distribution

```
1 print(df['Income'].value_counts())
```

>_ RESULT:

```
Low      204
Medium   197
High      99
```

Pollution Exposure

```
1 high_pollution = (df['Pollution'] == 'High').mean() * 100
2 print(high_pollution)
```

>_ RESULT:

70.4%

Disease Prevalence

```
1 prevalence = (df['LungDisease'] == 'Yes').mean() * 100
2 print(prevalence)
```

>_ RESULT:

52.8%

C. Association Analysis: Smoking vs Lung Disease

```
1 crosstab_smoking = pd.crosstab(df['Smoking'], df['
    LungDisease'])
2 print(crosstab_smoking)
3 chi2, p, dof, expected = chi2_contingency(crosstab_smoking)
4 print('Chi-square =', chi2)
5 print('p-value =', p)
```

>_ RESULT:

LungDisease	No	Yes
Smoking		
No	184	109
Yes	52	155

Chi-square = 67.59
p-value = 2.01e-16

i The extremely small p-value (2.01×10^{-16}) provides overwhelming evidence against independence.

D. Odds Ratio Calculation

```
1 oddsratio, p = fisher_exact([[155, 52], [109, 184]])
2 print('Odds Ratio =', oddsratio)
```

>_ RESULT:

Odds Ratio = 5.03

E. Logistic Regression Modeling

```
1 df2 = df.copy()
2 df2['Smoking'] = df2['Smoking'].map({'Yes':1, 'No':0})
3 df2['Pollution'] = df2['Pollution'].map({'High':1, 'Low':0})
4 df2['LungDisease'] = df2['LungDisease'].map({'Yes':1, 'No':0})
5
6 model = smf.logit('LungDisease ~ Smoking + Age + Pollution +
7                  C(Income)', data=df2).fit()
8 print(model.summary())
```

>_ RESULT:

Variable	Coef	p-value
Smoking	1.702	<0.001
Age	0.040	<0.001
Pollution	1.303	<0.001
Income(Low)	0.440	0.123
Income(Medium)	0.220	0.440

F. Model Performance

```
1 pred_prob = model.predict(df2)
2 auc = roc_auc_score(df2['LungDisease'], pred_prob)
3 pred_class = (pred_prob > 0.5).astype(int)
4 cm = confusion_matrix(df2['LungDisease'], pred_class)
5 accuracy = accuracy_score(df2['LungDisease'], pred_class)
6
7 print('AUC =', auc)
8 print('Confusion Matrix:\n', cm)
9 print('Accuracy =', accuracy)
```

>_ RESULT:

AUC = 0.787

Confusion Matrix:

[[156 80]

[64 200]]

Accuracy = 0.712

G. Conclusions & Public Health Implications

- **Primary Risk Factor:** Smoking (OR = 5.03) - smokers have 5× higher odds of lung disease
- **Secondary Factors:** Age and high pollution exposure significantly increase risk
- **Non-significant:** Income level shows no statistical effect
- **Model Performance:** 71.2% accuracy with AUC of 0.787 indicates moderate predictive power
- **Policy Recommendations:** Focus on smoking cessation programs and pollution control measures

PROBLEM 2

One-Way ANOVA: Exercise Programs and Weight Loss

Problem Statement

Determine if three different exercise programs (A, B, C) impact weight loss differently. Recruit 90 people, randomly assign 30 to each program. Response variable: weight loss (pounds).

Python Implementation

```

1  import numpy as np
2  import pandas as pd
3  from scipy.stats import f_oneway, shapiro, levene
4  from statsmodels.formula.api import ols
5  import statsmodels.api as sm
6  from statsmodels.stats.multicomp import pairwise_tukeyhsd
7
8  np.random.seed(42)
9  program_A = np.random.normal(5, 1.5, 30)
10 program_B = np.random.normal(7, 1.5, 30)
11 program_C = np.random.normal(5.5, 1.5, 30)
12
13 df = pd.DataFrame({
14     'WeightLoss': np.concatenate([program_A, program_B,
15     program_C]),
16     'Program': ['A']*30 + ['B']*30 + ['C']*30
17 })
18
19 model = ols('WeightLoss ~ Program', data=df).fit()
20 anova_table = sm.stats.anova_lm(model, typ=2)
21 print(anova_table)

```

>_ RESULT:

	sum_sq	df	F	PR(>F)
Program	67.416974	2.0	16.890178	6.342321e-07
Residual	173.629808	87.0	NaN	NaN

Assumption Validation

```
1 shapiro_p = stats.shapiro(model.resid)[1]
2 levene_p = stats.levene(df['WeightLoss'][df['Program']=='A'
3                           ],
4                           df['WeightLoss'][df['Program']=='B'],
5                           df['WeightLoss'][df['Program']=='C'])[1]
6 print(f"Shapiro-Wilk p-value: {shapiro_p:.4f}")
7 print(f"Levene's p-value: {levene_p:.4f}")
```

>_ RESULT:

Shapiro-Wilk p-value: 0.8948
Levene's p-value: 0.8627

💡 Key Insight

Both normality ($p = 0.895$) and homogeneity of variance ($p = 0.863$) assumptions are satisfied, validating the ANOVA results.

Post-hoc Analysis

>_ RESULT:

Multiple Comparison of Means - Tukey HSD, FWER=0.05

group1	group2	meandiff	p-adj	lower	upper	reject
A	B	2.1005	0.000	1.2307	2.9702	True
A	C	0.8015	0.0772	-0.0682	1.6713	False
B	C	-1.2989	0.0017	-2.1687	-0.4292	True

Interpretation

- **Program B** demonstrates significantly superior weight loss compared to both Program A ($p < 0.001$) and Program C ($p < 0.01$)
- **No significant difference** exists between Programs A and C ($p = 0.077$)
- **Recommendation:** Program B is the most effective exercise intervention

PROBLEM 3

Chi-Square Analysis on Nigeria Child Anemia Dataset

Data Source

2018 Nigeria Demographic and Health Surveys (NDHS) - Investigating factors affecting anemia levels among children aged 0-59 months.

Analysis Code

```
1 import pandas as pd
2 import numpy as np
3 from scipy.stats import chi2_contingency
4
5 df = pd.read_csv('children_anemia.csv')
6 clean_df = df.dropna(subset=['Wealth index combined', '
    Anemia level'])
7 contingency_table = pd.crosstab(clean_df['Wealth index
    combined'],
8                                 clean_df['Anemia level'])
9 chi2, p_val, dof, expected = chi2_contingency(
    contingency_table)
10
11 print(f"Chi-Square Statistic: {chi2:.2f}")
12 print(f"P-value: {p_val:.4e}")
```

>_ RESULT:

Chi-Square Statistic: 203.15
P-value: 7.2798e-37
Degrees of Freedom: 12

Contingency Table (Percentages)

>_ RESULT:

Anemia level	Mild	Moderate	Not anemic	Severe
Wealth index combined				
Middle	28.99	30.16	39.20	1.66
Poorer	27.44	32.52	38.12	1.92
Poorest	28.39	35.66	33.89	2.06
Richer	26.84	28.74	42.63	1.79
Richest	24.29	22.17	52.29	1.25

Hypothesis Testing

H_0 : Wealth index and child anemia level are independent

H_1 : Wealth index and child anemia level are associated

 **Key Insight**

The p-value (7.28×10^3) is astronomically small, providing conclusive evidence that wealth status is strongly associated with childhood anemia in Nigeria.

Key Findings

- **Poorest quintile:** Highest severe anemia rate (2.06%) and lowest healthy rate (33.89%)
- **Richest quintile:** Lowest severe anemia (1.25%) and highest healthy rate (52.29%)
- **Gradient effect:** Clear socioeconomic gradient exists across all anemia severity levels

Policy Implications

- Target poverty reduction as a primary intervention strategy
- Implement nutritional supplementation programs in low-income communities
- Improve healthcare access and maternal education in impoverished regions

PROBLEM 4

Fisher's Exact Test: Drug Treatments and Disease Outcome

Problem Context

Three drug treatments (A, B, C) are compared for disease outcomes. Test association between treatment and disease status, with post-hoc pairwise comparisons.

Contingency Data

Treatment	No Disease	Disease	Success Rate
Drug A	40	10	80%
Drug B	10	40	20%
Drug C	25	25	50%

Python Implementation

```
1 import numpy as np
2 from scipy.stats import chi2_contingency, fisher_exact
3 from statsmodels.stats.multitest import multipletests
4
5 table = np.array([[40, 10], [10, 40], [25, 25]])
6 chi2_stat, p_global, dof, expected = chi2_contingency(table)
7 print(f"Global P-value: {p_global:.4e}")
8
9 pairs = [(0,1), (0,2), (1,2)]
10 p_values = []
11 for i, j in pairs:
12     _, p_pair = fisher_exact(table[[i, j]])
13     p_values.append(p_pair)
14
15 reject, p_adjusted, _, _ = multipletests(p_values, method='
    bonferroni')
```

>_ RESULT:

```
Global P-value: 1.5230e-08
Drug A vs Drug B: Adjusted P = 0.0000 (Significant: True)
Drug A vs Drug C: Adjusted P = 0.0092 (Significant: True)
Drug B vs Drug C: Adjusted P = 0.0092 (Significant: True)
```

Key Insight

All pairwise comparisons remain statistically significant after Bonferroni correction, confirming that each drug has a distinctly different efficacy profile.

Clinical Interpretation

- **Drug A (80% success):** Most effective treatment option
- **Drug C (50% success):** Moderately effective, suitable as second-line
- **Drug B (20% success):** Least effective, not recommended for routine use
- All differences are statistically robust ($p < 0.01$ after correction)

PROBLEM 5

Logistic Regression GLM - Graduate School Admission

Research Question

How do GRE scores, GPA, and undergraduate institution prestige affect graduate school admission?

Model Fitting

```
1 import pandas as pd
2 import statsmodels.formula.api as smf
3
4 df = pd.read_csv('binary.csv')
5 model = smf.logit('admit ~ gre + gpa + C(rank)', data=df).
    fit()
6 print(model.summary())
```

Regression Results

>_ RESULT:

	coef	std err	z	P> z
Intercept	-3.9900	1.140	-3.500	0.000
C(rank) [T.2]	-0.6754	0.316	-2.134	0.033
C(rank) [T.3]	-1.3402	0.345	-3.881	0.000
C(rank) [T.4]	-1.5515	0.418	-3.713	0.000
gre	0.0023	0.001	2.070	0.038
gpa	0.8040	0.332	2.423	0.015

Odds Ratios

➤ RESULT:

Odds Ratios:

Intercept	0.0185
C(rank) [T.2]	0.5089
C(rank) [T.3]	0.2618
C(rank) [T.4]	0.2119
gre	1.0023
gpa	2.2345

💡 Key Insight

GPA is the strongest predictor - a one-point increase more than doubles admission odds (OR = 2.23). Institution rank shows dramatic effects, with Rank 4 students having only 21% the odds of Rank 1 students.

Practical Implications

- Academic performance (GPA) matters more than test scores (GRE)
- Prestigious undergraduate institutions provide significant advantage
- Marginal effect of GRE suggests diminishing returns beyond threshold

PROBLEM 6

Poisson Regression GLM - Number of Awards

Research Context

Modeling the number of academic awards earned by high school students based on program type and math exam scores.

Model Specification

```
1 import pandas as pd
2 import statsmodels.formula.api as smf
3
4 df = pd.read_csv('poisson_sim.csv')
5 model = smf.glm('num_awards ~ C(prog) + math', data=df,
6                 family=sm.families.Poisson()).fit()
7 print(model.summary())
```

Coefficient Estimates

>_ RESULT:

	coef	std err	z	P> z
Intercept	-5.2471	0.658	-7.969	0.000
C(prog) [T.2]	1.0839	0.358	3.025	0.002
C(prog) [T.3]	0.3698	0.441	0.838	0.402
math	0.0702	0.011	6.619	0.000

Incident Rate Ratios

>_ RESULT:

IRR:

Intercept	0.0053
C(prog) [T.2]	2.9561
C(prog) [T.3]	1.4475
math	1.0727

Substantive Interpretation

- **Math Score:** Each additional point increases expected awards by 7.3% (IRR = 1.073)
- **Vocational Program:** Students earn nearly $3\times$ more awards than Academic baseline
- **General Program:** Not significantly different from Academic ($p = 0.402$)

PROBLEM 7

Negative Binomial Regression - Days Absent

Research Question

What factors predict the number of days absent among high school juniors?

Model Fitting

```
1 import pandas as pd
2 import statsmodels.formula.api as smf
3
4 df = pd.read_stata('nb_data.dta')
5 model = smf.glm('daysabs ~ C(prog) + math', data=df,
6                 family=sm.families.NegativeBinomial()).fit()
7 print(model.summary())
```

Results

>_ RESULT:

	coef	std err	z	P> z
Intercept	2.6150	0.200	13.054	0.000
C(prog) [T.2]	-0.4408	0.185	-2.379	0.017
C(prog) [T.3]	-1.2786	0.203	-6.284	0.000
math	-0.0060	0.003	-2.358	0.018

Mean of daysabs: 5.9554

Variance of daysabs: 49.5187 (Overdispersion present)

Key Insight

Variance (49.5) greatly exceeds mean (6.0), confirming overdispersion and justifying the Negative Binomial over Poisson.

Key Predictors

- **Math Scores:** Higher scores associated with fewer absences (coefficient = -0.006, p = 0.018)

- **Program Type:** Academic track (baseline) shows best attendance
- General and Vocational students have significantly more absences

PROBLEM 8

Zero-Inflated Poisson Regression - Fish Catch Data

Problem Context

Modeling fish catch counts with excess zeros due to non-fishers and unsuccessful fishing attempts.

ZIP Model Implementation

```
1 import pandas as pd
2 import statsmodels.api as sm
3 from statsmodels.discrete.count_model import
   ZeroInflatedPoisson
4
5 df = pd.read_csv('fish.csv')
6 X = df[['camper', 'persons', 'child']]
7 X = sm.add_constant(X)
8 X_infl = df[['child', 'persons']]
9 X_infl = sm.add_constant(X_infl)
10
11 model = ZeroInflatedPoisson(df['count'], X, exog_infl=X_infl
   ).fit()
12 print(model.summary())
```

Model Results

>_ RESULT:

	coef	std err	z	P> z
inflate_const	0.9813	0.414	2.370	0.018
inflate_child	1.8227	0.315	5.780	0.000
inflate_persons	-0.8454	0.190	-4.456	0.000
const	-0.8452	0.166	-5.103	0.000
camper	0.7597	0.094	8.096	0.000
persons	0.8337	0.043	19.587	0.000
child	-1.1472	0.091	-12.539	0.000

Dual Interpretation

Non-Fisher Probability (Inflation Model)

>_ RESULT:

Odds Ratios (Being a Non-Fisher):

inflate_const	2.6679
inflate_child	6.1887
inflate_persons	0.4294

Catch Rate (Count Model)

>_ RESULT:

Incident Rate Ratios (Catch Rate):

const	0.4295
camper	2.1377
persons	2.3019
child	0.3175

Key Insight

Children have opposite effects on the two processes: groups with children are 6.19× more likely to be non-fishers, but when fishing occurs, each child reduces catch rate by 68%.

☰ PROBLEM 9

Zero-Inflated Poisson - Detailed Analysis

Extended ZIP Analysis

Alternative Specification

```

1  import pandas as pd
2  import statsmodels.api as sm
3  import statsmodels.formula.api as smf
4  import numpy as np
5
6  url = "https://stats.idre.ucla.edu/stat/data/fish.csv"
7  fish = pd.read_csv(url)
8  fish['child'] = fish['child'].astype(int)
9
10 zip_model = sm.ZeroInflatedPoisson.from_formula(
11     "count ~ persons + child + camper",
12     fish,
13     exog_infl=fish[['persons', 'child', 'camper']],
14     inflation='logit'
15 )
16 result = zip_model.fit(method='bfgs')
17 print(result.summary())

```

Detailed Output

>_ RESULT:

ZeroInflatedPoisson Regression Results

Dep. Variable: count
No. Observations: 250
Pseudo R-squ.: 0.276
Log-Likelihood: -1032.5

	coef	std err	z	P> z
inflate_persons	-0.45	0.16	-2.81	0.005
inflate_child	0.82	0.20	4.10	0.000
inflate_camper	-1.15	0.31	-3.70	0.000
Intercept	1.12	0.15	7.46	0.000
persons	0.38	0.05	7.60	0.000
child	-0.56	0.09	-6.22	0.000
camper	0.71	0.11	6.45	0.000

Marginal Effects

- **Persons:** $IRR = e^{0.38} 1.46346\%$ increase in expected catch per additional person
- **Children:** $IRR = e^{-0.56} 0.57343\%$ decrease in expected catch per child
- **Camper:** $IRR = e^{0.71} 2.033$ Double the catch rate compared to non – campers

PROBLEM 10

Zero-Truncated Poisson - Hospital Length of Stay

Study Context

Modeling hospital stay duration (minimum 1 day) by age, insurance type, and in-hospital mortality.

Model Specification

```
1 import pandas as pd
2 import numpy as np
3 import statsmodels.api as sm
4 import statsmodels.formula.api as smf
5
6 df = pd.read_csv('ztp_temp.csv')
7 model = smf.glm('stay ~ age + hmo + died', data=df,
8               family=sm.families.NegativeBinomial()).fit()
9 print(model.summary())
```

Results

>_ RESULT:

	coef	std err	z	P> z
Intercept	2.4378	0.091	26.920	0.000
age	-0.0147	0.016	-0.894	0.371
hmo	-0.1375	0.075	-1.845	0.065
died	-0.2038	0.058	-3.505	0.000

Incident Rate Ratios

>_ RESULT:

IRR:

Intercept	11.4473
age	0.9854
hmo	0.8715
died	0.8156

Clinical Interpretation

- **In-hospital mortality:** 18.4% shorter stays ($p < 0.001$)
- **HMO insurance:** 12.8% shorter stays (marginally significant, $p = 0.065$)
- **Age:** No significant effect ($p = 0.371$)

PROBLEM 11

Zero-Truncated Negative Binomial - Hospital Stay

Advanced Modeling: ZTNB

Custom Likelihood Implementation

```

1 from statsmodels.base.model import GenericLikelihoodModel
2 import numpy as np
3
4 class ZTNB(GenericLikelihoodModel):
5     def loglike(self, params):
6         beta = params[:-1]
7         alpha = params[-1]
8         if alpha <= 0: return -np.inf
9         mu = np.exp(np.dot(self.exog, beta))
10        size = 1 / alpha
11        prob = size / (size + mu)
12        ll_nb = nbinom.logpmf(self.endog, size, prob)
13        p0 = (1 + alpha * mu) ** (-1/alpha)
14        return np.sum(ll_nb - np.log(1 - p0))
15
16 model = ZTNB(df['stay'], X).fit()

```

ZTNB Results

>_ RESULT:

	coef	std err	z	P> z
const	2.4083	0.072	33.457	0.000
age	-0.0157	0.013	-1.198	0.231
hmo	-0.1471	0.059	-2.484	0.013
died	-0.2177	0.046	-4.717	0.000
alpha	0.5663	0.031	18.132	0.000

Incident Rate Ratios

>_ RESULT:

```
IRR:
const      11.1152
age         0.9844
hmo         0.8632
died        0.8043
```

Key Insight

The ZTNB model reveals HMO insurance as significant ($p = 0.013$), unlike the Poisson model. The alpha parameter (0.566, $p < 0.001$) confirms significant overdispersion, justifying the negative binomial approach.

Final Conclusions

- **Significant Predictors:** Mortality status and HMO insurance
- Patients who die have 19.6% shorter hospital stays
- HMO patients have 13.7% shorter stays than privately insured patients
- Age shows no statistically significant effect
- Zero-truncated negative binomial is the appropriate model due to:
 - Minimum stay of 1 day (zero truncation)
 - Significant overdispersion ($\alpha = 0.566$, $p < 0.001$)